



Security Audit

Defi Bull World 4th (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	17
Conclusion	18
Our Methodology	19
Disclaimers	21
About Hashlock	22

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Defi Bull World team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Defi Bull World is a cutting-edge digital asset and DeFi education platform built on the Flare Network with its native token FLR, led by seasoned blockchain experts since Bitcoin's inception. The platform delivers expert insights and regular updates through courses, webinars, and multimedia content, with plans to go multichain by Q3 2025. With new courses added monthly, AI-powered multilingual content, interactive experiences like sweepstakes and tests, and guaranteed long-term access for early members, Defi Bull World is committed to transparency through regular audits and public disclosure of contracts and wallets, while reinvesting revenues to drive sustainable growth.

Project Name: Defi Bull World

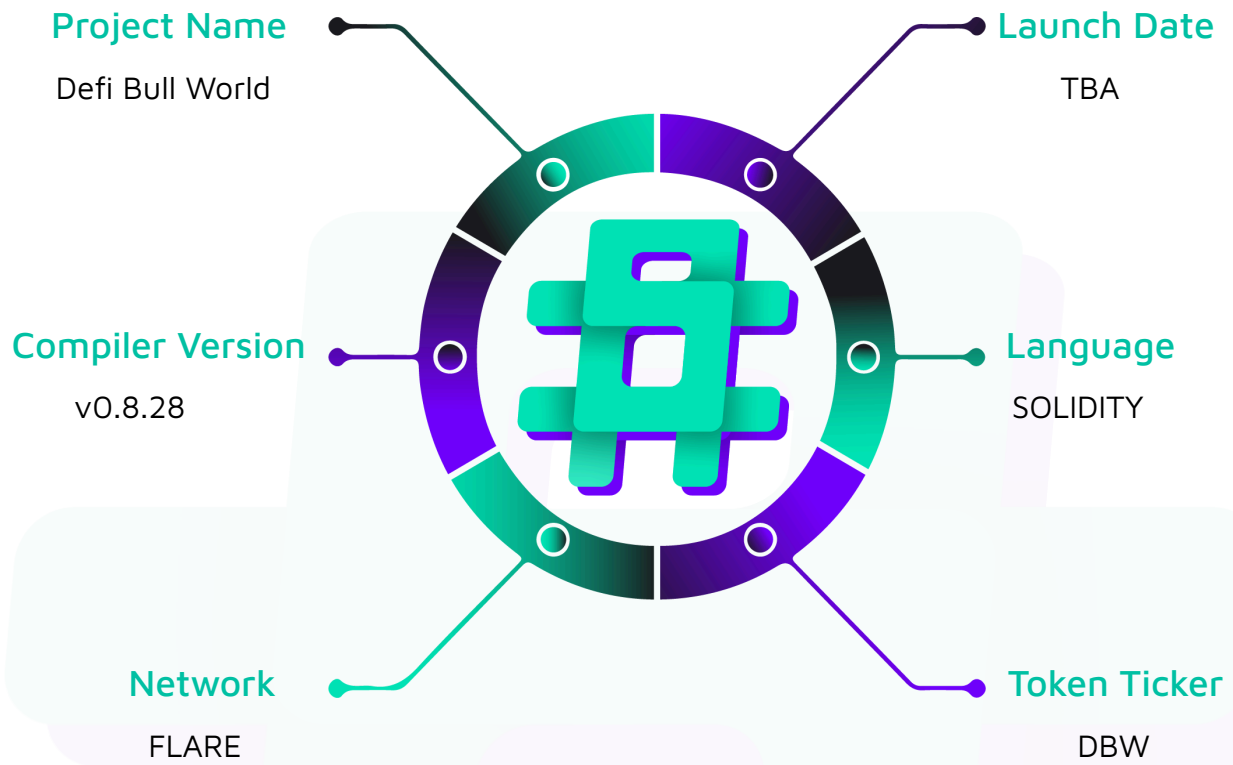
Project Type: DeFi

Compiler Version: 0.8.28

Website: <https://www.defibullworld.com/>

Logo:



Visualised Context:

Project Visuals:

Master DeFi Education

Unlock Your Financial Future

Join the revolution in decentralized finance education. Learn, earn, and grow.

[Start Learning & Earning →](#)
[View Membership](#)



Who We Are

We're not just an education platform – we're a movement dedicated to redefining how decentralized finance is learned and experienced.

Founded on the belief that knowledge is the foundation of empowerment, our platform bridges the gap between traditional finance and the new digital economy. With a team of industry experts and cutting-edge technology, we're committed to delivering a secure, transparent, and collaborative environment where everyone can thrive in the DeFi space.



Transparent Education

We deliver in-depth DeFi insights with clarity and integrity, empowering you with actionable knowledge.



Robust Security

Experience top-tier protection with state-of-the-art protocols that secure your digital journey.



Community Empowerment

Become part of a global network that values collaboration, support, and shared growth.



Data-Driven Insights

Leverage real-time market analysis and trends curated by industry experts to make informed decisions.

Audit Scope

We at Hashlock audited the solidity code within the Defi Bull World project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Defi Bull World Smart Contracts
Platform	Flare / Solidity
Audit Date	June, 2025
Contract 1	TestRewardsManager.sol
Contract 1 MD5 Hash	25d4c7eb2fb41e91d71aa3eb6968f32e

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 Medium severity vulnerability

1 Low severity vulnerability

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
TestRewardsManager.sol Allows authorized verifiers to: - Trigger the distribution of DBW token rewards directly to a user's address. - Update the contract's on-chain price data. Allows the owner to: - Set the reward amount, slippage tolerance, and other core parameters. - Manage authorized verifier and upgrader roles. - Pause and unpause all reward distributions. - Withdraw any surplus tokens from the contract	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Defi Bull World project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Defi Bull World project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] TestRewardsManager#getTWAPPrice - DOS in getTWAPPrice Function

Description

The `getTWAPPrice` function will revert and cause a transaction to fail if the only price observation within the `twapPeriod` was recorded in the same block as the function call.

Vulnerability Details

The issue is triggered when `getTWAPPrice` is called under a specific timing condition. When the function is called and there is only one price observation within the defined time window (`twapPeriod`) that was recorded in the same block as the current transaction, the function fails to calculate a proper time-weighted average. The main loop correctly identifies this single observation as valid and sets `foundValidObs = true`, but because there is no second observation to measure against, the `totalWeight` remains zero. After the loop, the code attempts to add weight for the final time period using the condition `if (foundValidObs && prevTimestamp > 0 && currentTime > prevTimestamp)`. However, since both `currentTime` and `prevTimestamp` are identical (same block timestamp), the condition `currentTime > prevTimestamp` evaluates to false, causing the code block to be skipped. This leaves `totalWeight` at zero, which triggers the final check `if (totalWeight == 0) revert InvalidPriceData();` causing the function to revert.

```
// Problematic code section

if (foundValidObs && prevTimestamp > 0 && currentTime > prevTimestamp) {

    uint256 lastTimeDelta = currentTime - prevTimestamp;

    if (lastTimeDelta <= twapPeriod) {

        weightedPriceSum += prevPrice * lastTimeDelta;

        totalWeight += lastTimeDelta;
    }
}
```



```
    }  
}  
  
if (totalWeight == 0) revert InvalidPriceData();
```

Impact

When users attempt to claim rewards through the `claimReward` function, which depends on `getTWAPPrice` for calculating reward amounts, the transaction will fail with `InvalidPriceData()` error even though valid price data exists.

Recommendation

Modify the final check in the `getTWAPPrice` function to handle the edge case gracefully. Instead of immediately reverting when `totalWeight` is zero, first verify if a valid observation was found. If one exists, return its price as a spot price rather than failing the transaction.

```
if (totalWeight == 0) {  
    if (foundValidObs) {  
        return prevPrice;  
    }  
    revert InvalidPriceData();  
}
```

Status

Resolved

Low

[L-01] Contracts - Dead Code: Redundant Slippage Check in `claimReward` Function

Description

In the `claimReward` function, there is a section of code designed to handle cases where the contract's token balance might be lower than the expected reward amount.

```
// Enhanced balance checks

uint256 contractBalance = rewardToken.balanceOf(address(this));

if (contractBalance < minDBWAmount) revert InsufficientBalance();


// Transfer the expected amount or contract balance if lower, but never less than
minimum

uint256 actualAmount = expectedDBWAmount;

if (contractBalance < expectedDBWAmount) {

    actualAmount = contractBalance;

    if (actualAmount < minDBWAmount) revert SlippageTooHigh();

}
```

The function first correctly checks if the contract has enough tokens to cover the minimum required amount (`minDBWAmount`). If this check passes, the code continues. Later, within a block that handles low balances, it re-checks if the balance is less than the minimum required amount. However, this second check can never be true, because the function would have already failed earlier if the balance was insufficient. While this does not pose a security risk or cause financial loss, it does reduce code quality.

Recommendation

Remove the redundant and unreachable if statement that checks if (`actualAmount < minDBWAmount`) from the `claimReward` function.

Status

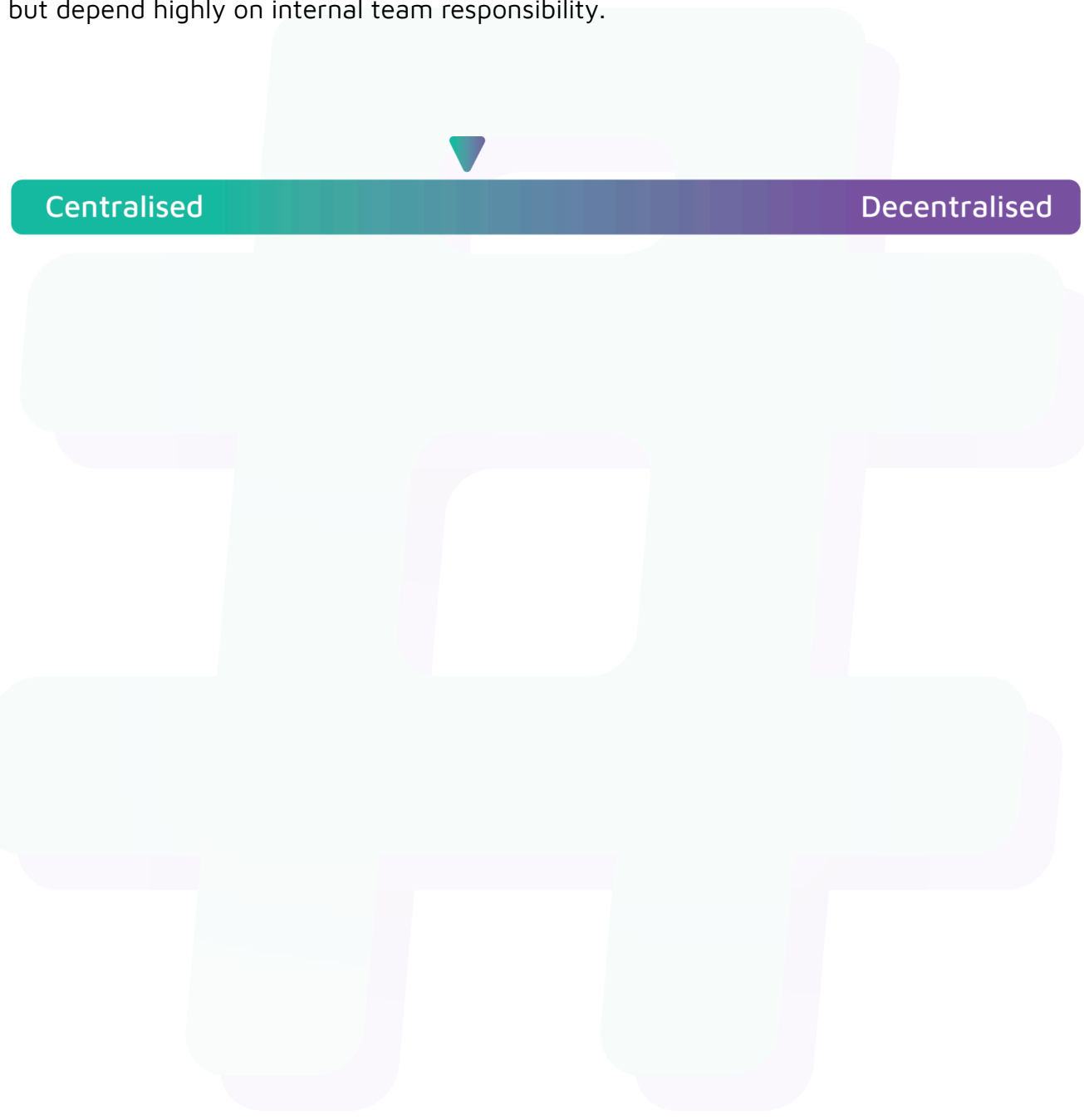
Resolved



Centralisation

The Defi Bull World project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Defi Bull World project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.